

Question 15 – Multi-Source Log Processing System

Aim

Solve $T(n) = 2T(n/2) + \log n$ using the Substitution Method. Expand the recurrence, derive the complexity, and analyze performance for large log volumes.

Algorithm

1. Read n .
2. Apply the recurrence recursively.
3. Compute additional work.
4. Print the estimated cost.

Source Code

```
import math

def T(n):
    if n <= 1:
        return 1

    return 2 * T(n // 2) + math.log2(n)

n = int(input("Enter value of n: "))

result = T(n)

print("\nApplication : Multi-Source Log Processing System")
print("Recurrence :  $T(n) = 2T(n/2) + \log n$ ")
print("Estimated Cost :", round(result,2))
```

A screenshot of a code editor interface. The left pane shows a Python file named 'main.py' with the following code:

```
1 import math
2
3 def T(n):
4     if n <= 1:
5         return 1
6
7     return 2 * T(n // 2) + math.log2(n)
8
9 n = int(input("Enter value of n: "))
10
11 result = T(n)
12
13 print("\nApplication : Multi-Source Log Processing System")
14 print("Recurrence : T(n) = 2T(n/2) + log n")
15 print("Estimated Cost :", round(result,2))
```

The right pane shows the output of the program:

```
Enter value of n: 3
Application : Multi-Source Log Processing System
Recurrence : T(n) = 2T(n/2) + log n
Estimated Cost : 3.58
=== Code Execution Successful ===
```

Analysis

Expansion leads to linear growth.

Final Time Complexity: $\Theta(n)$.

Space Complexity: $O(\log n)$.

Conclusion

The implementation demonstrates the recurrence and supports the theoretical complexity analysis.